HW4.3 Data Visualizations QMD

STAT 184

Aryan Deshmukh

2026-04-16

Busiest Airports Analysis

Introduction

This report examines the passenger traffic trends of 6 major international airports, namely ATL, FRA, PKX, DFW, DXB and LHR, from 2020 to 2025 using data from Wikipedia. The report highlights the trends using line graphs made using the ggplot2 package to present yearly passenger traffic changes, specifically the change of passenger numbers over time and between the 6 airports.

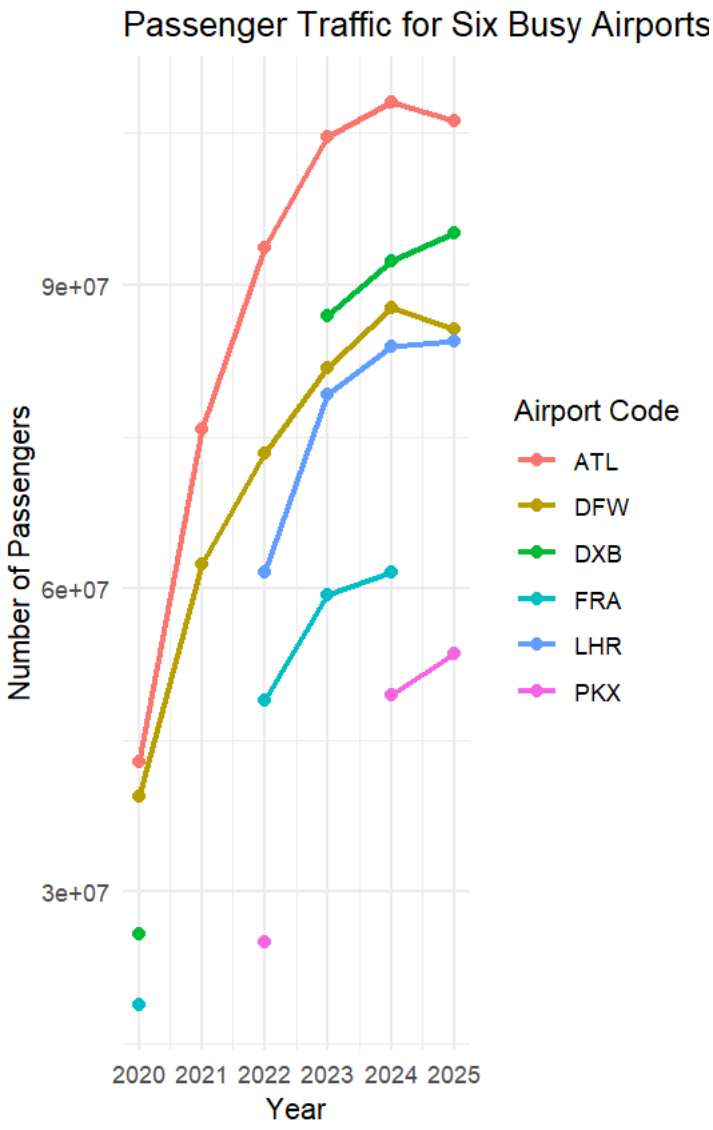
Airport Table

Table 1

	code	airport	`2020`	`2021`	`2022`	`2023`	`2024`	`2025`
	<fct>	<fct>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	ATL	Hartsfield-Jackson Atlanta International Airport	42918685	75704760	93699630	104653451	108067766	106302208
2	DFW	Dallas Fort Worth International Airport	39364990	62465756	73362946	81755538	87817864	85660127
3	DXB	Dubai International Airport	25836771	NA	NA	86994365	92331506	95200000
4	FRA	Frankfurt Airport	18768601	NA	48918482	59355389	61564957	NA
5	LHR	Heathrow Airport	NA	NA	61614508	79183364	83884572	84463000
6	PKX	Beijing Daxing International Airport	NA	NA	25051012	NA	49441029	53618949

Airport Plot

Figure 1



Analysis

The table and line graph together help compare passenger traffic across six major international airports: ATL, DFW, DXB, FRA, LHR, and PKX, from 2020 to 2025. The table shows the

exact passenger numbers for each airport each year, while the graph makes it easier to see the overall trends and how each airport changes over time.

Overall, most airports show a clear increase in passenger traffic after 2020, but the amount of growth is different for each one. Hartsfield–Jackson Atlanta International Airport (ATL) has the highest passenger numbers in every year, growing from about 42.9 million in 2020 to over 108 million in 2024, before slightly dropping in 2025. Dallas Fort Worth International Airport (DFW) follows a similar pattern, steadily increasing from about 39.4 million in 2020 to around 87.8 million in 2024, with a small decrease in 2025.

Dubai International Airport (DXB) has some missing data, but from the years available, it shows strong growth, especially from 2023 to 2025, going from about 86.9 million to 95.2 million passengers. Frankfurt Airport (FRA) also increases over time, but more slowly compared to the others, going from about 18.8 million in 2020 to around 61.6 million in 2024, with some missing years.

Heathrow Airport (LHR) shows steady growth after 2022, increasing from about 61.6 million passengers to 84.4 million in 2025. Beijing Daxing International Airport (PKX) has several missing years, but the available data still shows an overall increase from about 25.1 million in 2022 to 53.6 million in 2025.

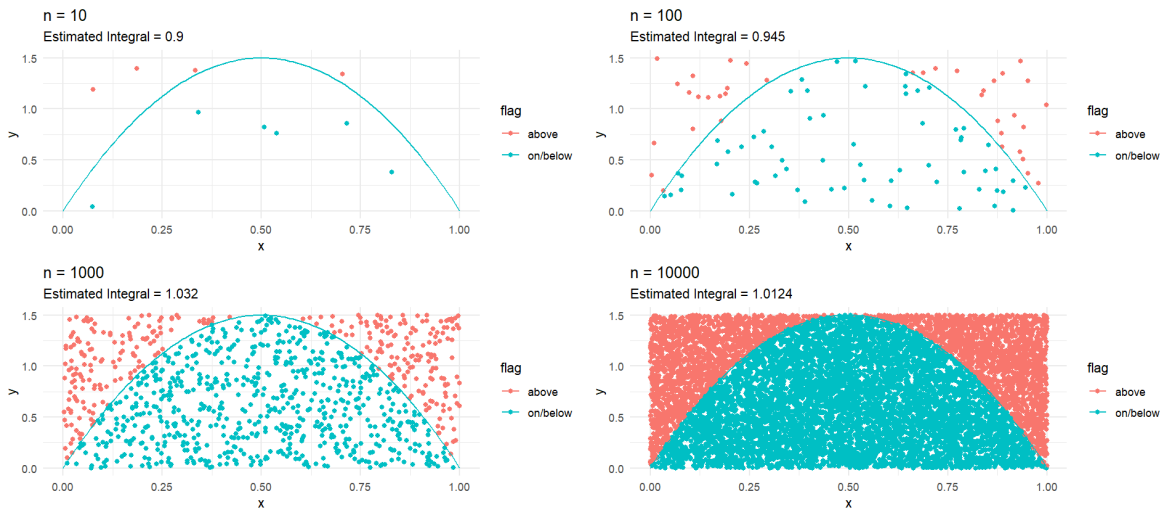
In conclusion, most airports show recovery and growth after 2020, with ATL and DFW being the busiest overall. DXB and LHR also show strong increases, FRA grows more slowly, and PKX is harder to fully compare because of missing data.

Monte Carlo Numerical Integration

This analysis of the Monte Carlo Integration is to estimate the value of an integral using random points, the sample sizes used for the estimates were 10, 100, 1000 and 10000. The main purpose is to see the accuracy of the estimate at different sample sizes as we increase them. The idea used is to generate random points in a set region and check whether they lie below or above the curve, using this method we can estimate the area under the curve, the process is repeated at different sample sizes and then compared. The comparison is done through multiple plots for each sample size.

Monte Carlo Integration Plot

Figure 2



Analysis

This figure shows four scatterplots of randomly generated points used to estimate a Monte Carlo integral for the Beta (2,2) distribution on the interval [0,1]. Panels A, B, C, and D represent sample sizes of 10, 100, 1,000, and 10,000 points. In each panel, the beta (2,2) density curve is shown along with the sampled points, and the points are coloured based on whether they fall above or below the curve.

Each panel also shows an estimate of the integral based on the proportion of points under the curve compared to the total number of points. Panel A (with 10 points) has a lot of variation and is not very reliable because the sample size is so small. As the sample size increases in Panels B, C, and D, the points start to fill the space more evenly, and the estimates become more accurate.

Overall, the results show that larger sample sizes lead to more stable and reliable Monte Carlo estimates. Panels C and D, especially, are much closer to the true value compared to the smaller samples.

Planning and Prompting GenAI Tools

Introduction

We will analyse the difference of generative AI's capability based on the quality of the prompts provided. For testing we have used a data set and given the task of tidying it and generate a data visualization plot to AI. The AI of choice used was OpenAI's ChatGPT, first we structured a SMART plan with goals, sub-goals and steps and generated a prompt with it to give to the AI and store it's response. Then from a previous response we had given a generic prompt and aksed to generate a graph.

The Full Plan

Goal

- Clean (tidy) the calcium.csv data
- Turn it into a usable format for analysis
- Make a plot that compares ulnar calcium levels over time
- Compare the null group vs the diet/exercise group across 4 years

Sub-goals / Checks

- Make sure each woman has 4 time points
- Make sure groups are correctly labeled
- Make sure no columns are mixed up after reshaping
- Check that the plot clearly shows two different trends

Steps

1. Load the dataset into R
2. Look at how the data is structured
3. Notice that the data is in wide format
4. Reshape it into long (tidy) format
5. Combine time columns into one time variable
6. Separate or label the two treatment groups
7. Make sure calcium values are numeric
8. Check for missing or messy values
9. Create a line plot using ggplot
10. Put time on x-axis and calcium on y-axis
11. Color lines by group

12. Compare trends between the two groups

ChatGPT's Response to Plan

This is a solid plan already—you're basically describing a clean data workflow + visualization pipeline. I'll tighten it into something you can actually *run in R*, while keeping your structure.

Step-by-step in **R (with code)**

1. Load the dataset

```
library(tidyverse)

calcium <- read.csv("calcium.csv")
```

---

2. Inspect the structure

```
str(calcium)
head(calcium)
```

You're checking:

- Are time columns like `year1, year2, year3, year4`?
- Is there a group column (e.g., `group`, `treatment`)?

3–5. Convert from **wide → long format**

This is the key step.

```
calcium_long <- calcium %>%
  pivot_longer(
    cols = starts_with("year"),   # adjust if column names differ
    names_to = "time",
    values_to = "calcium"
  )
```

---

6. Clean / label groups

Make sure groups are clear:

```
calcium_long$group <- factor(calcium_long$group,
                             levels = c("null", "diet_exercise"))
```

(Adjust names if needed.)

---

7. Ensure calcium is numeric

```
calcium_long$calcium <- as.numeric(calcium_long$calcium)
```

---

8. Check for issues

```
summary(calcium_long)
any(is.na(calcium_long$calcium))
```

Optional deeper check:

```
calcium_long %>% count(id)   # each woman should have 4 rows
```

9–12. Create the plot (ggplot)

```
ggplot(calcium_long, aes(x = time, y = calcium, group = id, color = group)) +
  geom_line(alpha = 0.5) +
  stat_summary(aes(group = group), fun = mean, geom = "line", size = 1.5) +
  labs(
    title = "Calcium Levels Over Time",
    x = "Time",
    y = "Calcium Level",
    color = "Group"
  ) +
  theme_minimal()
```

What this plot does (important for your write-up)

- Thin lines → individual women
- Thick lines → **average trend per group**
- Colors → compare:
  - Null group
  - Diet/exercise group

Your Sub-goals (checked properly)

- Each woman has 4 time points → `count(id)`
- Groups labeled → `factor()`
- No reshaping errors → check `head(calcium_long)`
- Clear comparison → mean trend lines

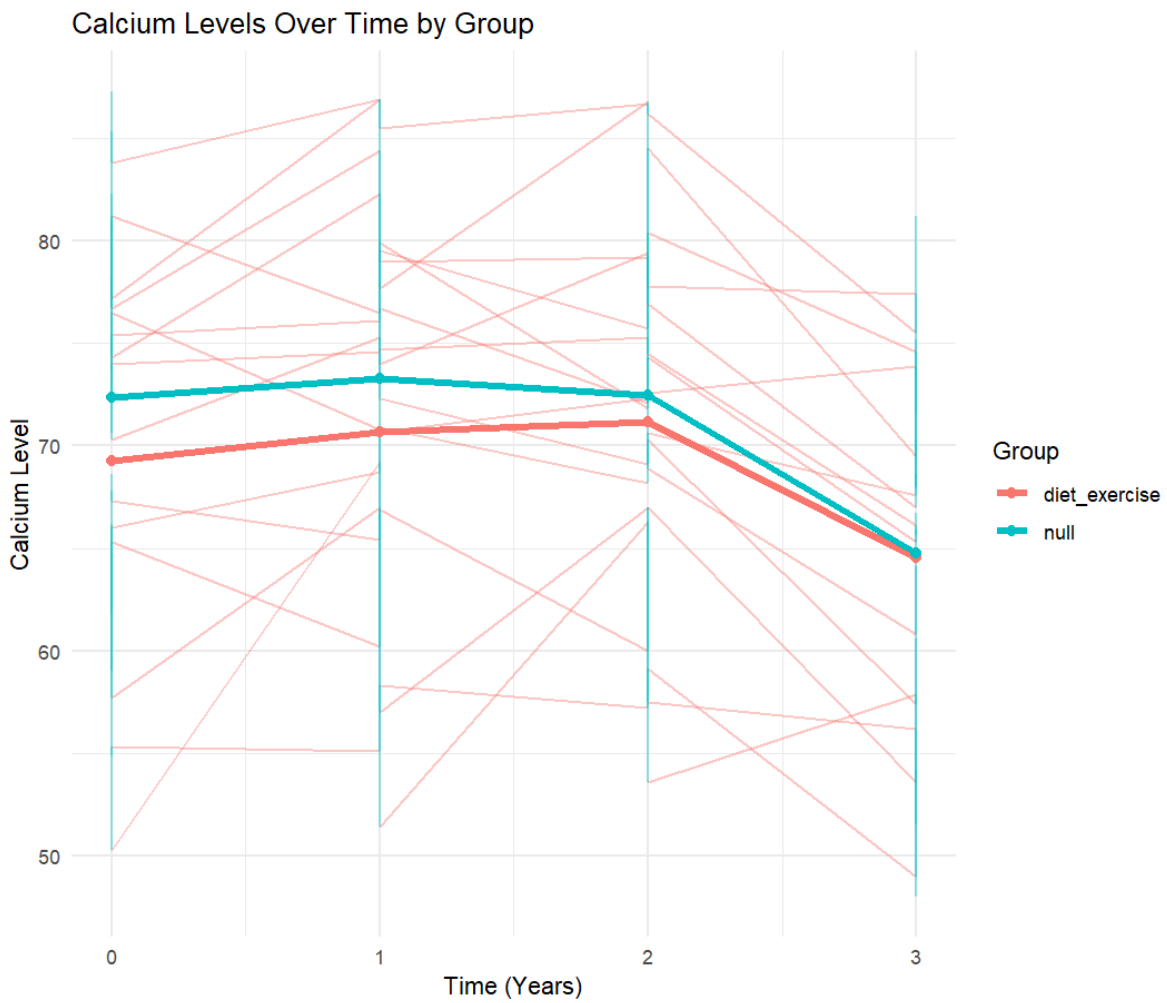
If you want it cleaner (better for grading)

Instead of individual lines (which can look messy), you could do:

```
ggplot(calcium_long, aes(x = time, y = calcium, color = group)) +
  stat_summary(fun = mean, geom = "line", size = 1.5) +
  stat_summary(fun = mean, geom = "point") +
  labs(title = "Average Calcium Levels by Group Over Time") +
  theme_minimal()
```

Data Visualisaton with ChatGPT's Generated Code

Figure 3

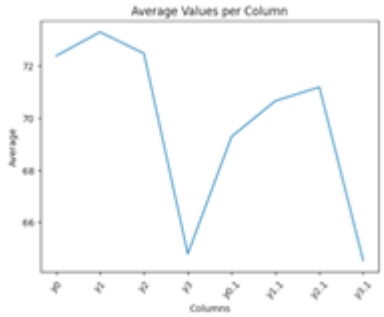


Generic Prompt

Create a data visualization for these data. Calcium.csv

Data Visualization with Generic Prompt

Figure 4



Analysis

The two graphs show a big difference mainly because of how the prompts were written. The graph made from the generic prompt is very basic and doesn't give much useful information. It only shows average values for each column, so you can't really tell what the data represents or how it changes over time. There are no clear labels for groups, and it doesn't compare anything specific. Because of that, the graph doesn't really tell any details; it just shows numbers in a simple way.

In contrast, the graph made with the full plan is much more detailed and meaningful. It clearly compares two groups (diet/exercise vs. null) over four years, which makes it easier to see trends and differences. The plan helped organise the data properly and made sure everything was labelled and structured correctly. As a result, the graph shows how calcium levels change over time and allows for real comparisons between the groups. Overall, the full plan leads to a clearer, more informative visualisation, while the generic prompt results in something more unclear and less useful.

Reflection

This assignment helped me understand the importance of both planning and using Quarto. I learnt that having a clear plan first can make the whole process easier, faster, and more organised. Breaking the task into goals, steps, and checks made it easier to understand what needed to be done and helped avoid mistakes. The planning section also showed me how important good instructions are when using AI tools. A detailed plan gave much better results than a simple or generic prompt. When the task was explained clearly, the graph was more accurate, more useful, and easier to understand. This taught me that AI works best when the user already has a strong idea of what they want to do. Another major part of this assignment

was learning how useful Quarto is. Quarto made it possible to put my code, graphs, tables, explanations, and appendices all into one professional document. Instead of having everything in separate files, I could keep the whole project organised in one place. It also made the final report look cleaner and easier to read. Overall, this assignment taught me that strong planning and good organisation are just as important as coding itself. Quarto is a very helpful tool because it turns data analysis into a complete report, and planning makes sure the report is clear and successful from the start.

GenAI Appendix

QuillBot

Quillbot was used in texts to check for Grammar and Spelling mistakes

Date: 16th April, 2026

Prompts: The paragraphs were given and it evaluated if any changes be made

Reponse: Corrections such as punctuation and spelling

ChatGPT

ChatGPT was used for the "Planning and Prompting GenAI Tools" section for generating the plot using the plan provided and for fixing errors with Quarto

Date: 16th April, 2026

Prompt: ### Goal

- Clean (tidy) the calcium.csv data
- Turn it into a usable format for analysis
- Make a plot that compares ulnar calcium levels over time
- Compare the null group vs the diet/exercise group across 4 years

  ```
  ### Sub-goals / Checks
  ```

- Make sure each woman has 4 time points
- Make sure groups are correctly labeled
- Make sure no columns are mixed up after reshaping
- Check that the plot clearly shows two different trends

Steps

1. Load the dataset into R
2. Look at how the data is structured
3. Notice that the data is in wide format
4. Reshape it into long (tidy) format
5. Combine time columns into one time variable
6. Separate or label the two treatment groups
7. Make sure calcium values are numeric
8. Check for missing or messy values
9. Create a line plot using ggplot
10. Put time on x-axis and calcium on y-axis
11. Color lines by group
12. Compare trends between the two groups

Response: This is a solid plan already—you're basically describing a clean data workflow + visualization pipeline. I'll tighten it into something you can actually *run in R*, while keeping your structure.

Step-by-step in **R (with code)**

1. Load the dataset

```
library(tidyverse)

calcium <- read.csv("calcium.csv")
```

---

2. Inspect the structure

```
str(calcium)
head(calcium)
```

You're checking:

- Are time columns like `year1, year2, year3, year4`?
- Is there a group column (e.g., `group`, `treatment`)?

3–5. Convert from **wide → long format**

This is the key step.

```
calcium_long <- calcium %>%
  pivot_longer(
    cols = starts_with("year"),   # adjust if column names differ
    names_to = "time",
    values_to = "calcium"
  )
```

---

6. Clean / label groups

Make sure groups are clear:

```
calcium_long$group <- factor(calcium_long$group,
                             levels = c("null", "diet_exercise"))
```

(Adjust names if needed.)

---

7. Ensure calcium is numeric

```
calcium_long$calcium <- as.numeric(calcium_long$calcium)
```

---

8. Check for issues

```
summary(calcium_long)
any(is.na(calcium_long$calcium))
```

Optional deeper check:

```
calcium_long %>% count(id)   # each woman should have 4 rows
```

9–12. Create the plot (ggplot)

```r
ggplot(calcium_long, aes(x = time, y = calcium, group = id, color = group)) +
  geom_line(alpha = 0.5) +
  stat_summary(aes(group = group), fun = mean, geom = "line", size = 1.5) +
  labs(
    title = "Calcium Levels Over Time",
    x = "Time",
    y = "Calcium Level",
    color = "Group"
  ) +
  theme_minimal()
```

What this plot does (important for your write-up)

- Thin lines → individual women
- Thick lines → **average trend per group**
- Colors → compare:
  - Null group
  - Diet/exercise group

Your Sub-goals (checked properly)

- Each woman has 4 time points → `count(id)`
- Groups labeled → `factor()`
- No reshaping errors → check `head(calcium_long)`
- Clear comparison → mean trend lines

If you want it cleaner (better for grading)

Instead of individual lines (which can look messy), you could do:

```r
ggplot(calcium_long, aes(x = time, y = calcium, color = group)) +
  stat_summary(fun = mean, geom = "line", size = 1.5) +
  stat_summary(fun = mean, geom = "point") +
  labs(title = "Average Calcium Levels by Group Over Time") +
  theme_minimal()
```

Prompt: I wanted to ask is for some reason my ## and ### shows up as text which i dont understand why

Response:

Most likely cause your headings are being treated as **plain text instead of Markdown headings**. In Quarto, `##` and `###` only work when they're on their own line in normal text—not inside code blocks or attached to other text.

Common Reasons + Fixes

1. You wrote heading text on the same line as a paragraph

❌ Wrong:

Markdown

```
Some paragraph text ## Analysis
```

✅ Correct:

Markdown

```
Some paragraph text.

## Analysis
```

You need a blank line before headings.

Code Appendix

```r
## Busiest Airports Wrangling Code

library(tidyverse)

airports_tidy <- data.frame(
  year = c(
    2020, 2021, 2022, 2023, 2024, 2025,
    2020, 2021, 2022, 2023, 2024, 2025,
```

```r
    2020, 2021, 2022, 2023, 2024, 2025,
    2020, 2021, 2022, 2023, 2024, 2025,
    2020, 2021, 2022, 2023, 2024, 2025,
    2020, 2021, 2022, 2023, 2024, 2025
  ),
  airport = c(
    "Hartsfield-Jackson Atlanta International Airport",
    "Hartsfield-Jackson Atlanta International Airport",
    "Hartsfield-Jackson Atlanta International Airport",
    "Hartsfield-Jackson Atlanta International Airport",
    "Hartsfield-Jackson Atlanta International Airport",
    "Hartsfield-Jackson Atlanta International Airport",

    "Frankfurt Airport",
    "Frankfurt Airport",
    "Frankfurt Airport",
    "Frankfurt Airport",
    "Frankfurt Airport",
    "Frankfurt Airport",

    "Beijing Daxing International Airport",
    "Beijing Daxing International Airport",
    "Beijing Daxing International Airport",
    "Beijing Daxing International Airport",
    "Beijing Daxing International Airport",
    "Beijing Daxing International Airport",

    "Dallas Fort Worth International Airport",
    "Dallas Fort Worth International Airport",
    "Dallas Fort Worth International Airport",
    "Dallas Fort Worth International Airport",
    "Dallas Fort Worth International Airport",
    "Dallas Fort Worth International Airport",

    "Dubai International Airport",
    "Dubai International Airport",
    "Dubai International Airport",
    "Dubai International Airport",
    "Dubai International Airport",
    "Dubai International Airport",

    "Heathrow Airport",
```

```
    "Heathrow Airport",
    "Heathrow Airport",
    "Heathrow Airport",
    "Heathrow Airport",
    "Heathrow Airport"
  ),

  code = c(
    "ATL", "ATL", "ATL", "ATL", "ATL", "ATL",
    "FRA", "FRA", "FRA", "FRA", "FRA", "FRA",
    "PKX", "PKX", "PKX", "PKX", "PKX", "PKX",
    "DFW", "DFW", "DFW", "DFW", "DFW", "DFW",
    "DXB", "DXB", "DXB", "DXB", "DXB", "DXB",
    "LHR", "LHR", "LHR", "LHR", "LHR", "LHR"
  ),

  passengers = c(
    42918685, 75704760, 93699630, 104653451, 108067766, 106302208,
    18768601, NA, 48918482, 59355389, 61564957, NA,
    NA, NA, 25051012, NA, 49441029, 53618949,
    39364990, 62465756, 73362946, 81755538, 87817864, 85660127,
    25836771, NA, NA, 86994365, 92331506, 95200000,
    NA, NA, 61614508, 79183364, 83884572, 84463000
  )
)

airports_tidy <- airports_tidy %>%
  mutate(
    year = as.integer(year),
    airport = as.factor(airport),
    code = as.factor(code),
    passengers = as.numeric(passengers)
  ) %>%
  arrange(code, year)

airports_tidy

## Busiest Airports Table Code

library(tidyverse)
airports_table <- airports_tidy %>%
  select(code, airport, year, passengers) %>%
```

```r
  pivot_wider(
    names_from = year,
    values_from = passengers
  ) %>%
  arrange(code)

airports_table

## Busiest Airports Plot Code

library(tidyverse)
ggplot(data = airports_tidy, aes(x = year, y = passengers, color = code, group = code)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    title = "Passenger Traffic for Six Busy Airports (2020-2025)",
    x = "Year",
    y = "Number of Passengers",
    color = "Airport Code"
  ) +
  scale_x_continuous(breaks = 2020:2025) +
  theme_minimal()

## Monte Carlo Simulation Code

library(tidyverse)

# Function
f <- function(x) {
  -(x - 4)^2 + 4
}

# Bounds
x_min <- 2
x_max <- 6
y_min <- 0
y_max <- 4

# Generate random points
monte_carlo_points <- function(n, x_min, x_max, y_min, y_max) {
  data.frame(
    x = runif(n, x_min, x_max),
```

```r
    y = runif(n, y_min, y_max)
  )
}

# Core simulation function
make_mc_data <- function(n) {

  mc_points <- monte_carlo_points(n, x_min, x_max, y_min, y_max)

  mc_points <- mc_points %>%
    mutate(
      fx = f(x),
      flag = if_else(y <= fx, "on/below", "above")
    )

  rectangle_area <- (x_max - x_min) * (y_max - y_min)

  estimate <- mean(mc_points$flag == "on/below") * rectangle_area

  mc_points %>%
    mutate(
      sample_size = paste("n =", n),
      estimate = round(estimate, 4),
      panel_label = paste(
        "n =", n,
        "\nEst. Numerical Integration:",
        round(estimate, 4)
      )
    )
}

# Run simulations
mc_10 <- make_mc_data(10)
mc_100 <- make_mc_data(100)
mc_1000 <- make_mc_data(1000)
mc_10000 <- make_mc_data(10000)

mc_all <- bind_rows(mc_10, mc_100, mc_1000, mc_10000)

# Curve data
curve_data <- data.frame(
  x = seq(x_min, x_max, length.out = 500)
```

```r
) %>%
  mutate(y = f(x))

# Plot
ggplot(mc_all, aes(x = x, y = y, color = flag)) +
  geom_point(alpha = 0.6) +
  geom_line(
    data = curve_data,
    aes(x = x, y = y),
    inherit.aes = FALSE,
    color = "blue"
  ) +
  facet_wrap(~ panel_label) +
  labs(
    title = "Monte Carlo Integration Example",
    x = "x",
    y = "y",
    color = "Region"
  ) +
  theme_minimal()

## Monte Carlo Numerical Integration; Wrangling and Plot Code

library(tidyverse)

mc_points <- data.frame(
  x = runif(1000, 0, 1),
  y = runif(1000, 0, 1.5)
)

mc_points <- mc_points %>%
  mutate(
    fx = dbeta(x, shape1 = 2, shape2 = 2),
    flag = if_else(y <= fx, "on/below", "above")
  )

rectangle_area <- (1 - 0) * (1.5 - 0)

estimate <- mean(mc_points$flag == "on/below") * rectangle_area

ggplot(mc_points, aes(x = x, y = y, color = flag)) +
  geom_point() +
```

```r
  stat_function(
    fun = dbeta,
    args = list(shape1 = 2, shape2 = 2),
    xlim = c(0, 1),
    color = "blue"
  ) +
  labs(
    title = "Monte Carlo Integration (n = 1000)",
    subtitle = paste("Estimated Integral =", round(estimate, 4)),
    x = "x",
    y = "y",
    color = "Region"
  ) +
  theme_minimal()

## ChatGPT's Generated SMART Plan Plot Code

library(tidyverse)

# Load data
calcium <- read.csv("calcium.csv")

# Add an ID column (since dataset doesn't have one)
calcium$id <- 1:nrow(calcium)

# Split into two groups
null_group <- calcium %>%
  select(id, y0, y1, y2, y3) %>%
  mutate(group = "null")

treatment_group <- calcium %>%
  select(id, y0.1, y1.1, y2.1, y3.1) %>%
  rename(
    y0 = y0.1,
    y1 = y1.1,
    y2 = y2.1,
    y3 = y3.1
  ) %>%
  mutate(group = "diet_exercise")

# Combine both groups
calcium_clean <- bind_rows(null_group, treatment_group)
```

```r
# Convert to long format
calcium_long <- calcium_clean %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "time",
    values_to = "calcium"
  )

# Clean time variable (y0 → 0, y1 → 1, etc.)
calcium_long$time <- as.numeric(gsub("y", "", calcium_long$time))

# Make group a factor
calcium_long$group <- factor(calcium_long$group)

# Plot
ggplot(calcium_long, aes(x = time, y = calcium, group = id, color = group)) +
  geom_line(alpha = 0.4) +
  stat_summary(aes(group = group), fun = mean, geom = "line", size = 1.5) +
  stat_summary(aes(group = group), fun = mean, geom = "point", size = 2) +
  labs(
    title = "Calcium Levels Over Time by Group",
    x = "Time (Years)",
    y = "Calcium Level",
    color = "Group"
  ) +
  theme_minimal()
```